TASK 02: EXPLORE VALUES AND TYPES IN THE INTERPRETER

Course: CSA08 – Python Programming
Assignment: 1 – Payment Calculator for a Village Milk Co-operative
Bloom's Level: K4 – Analyse
Marks: 4

1. Aim

To explore Python data types, arithmetic operators and floating-point behaviour using the Python interpreter, and understand why accurate money calculations are important in the milk payment system.

2. Python Data Types Using type()

The type() function identifies the data type of a value.

Interactive Session

```
>>> type(10)
<class 'int'>

>>> type(10.0)
<class 'float'>

>>> type("10")
<class 'str'>

>>> type(True)
<class 'bool'>

>>> type([10, 8.5])
<class 'list'>
```

Explanation

Value	Data type	Explanation
10	int	Whole number
10.0	float	Decimal number
"10"	str	Text enclosed in quotation marks

True	bool	Boolean value
[10, 8.5]	list	Collection of values

These types are useful in the milk payment system because litres and percentages can contain decimal values, farmer IDs can be stored as text, and multiple entries can be stored in a list.

3. Arithmetic Operators

Python supports different arithmetic operators.

Interactive Session

```
>>> 7 / 2
3.5


>>> 7 // 2
3


>>> 7 % 2
1


>>> -7 // 2
-4
```

Explanation

- / – Normal division. It returns a floating-point result.
- // – Floor division. It returns the quotient rounded down to the nearest whole number.
- % – Modulus. It returns the remainder.
- -7 // 2 gives -4 because floor division rounds down towards negative infinity, not towards zero.

4. Floating-Point Precision

Interactive Session

```
>>> 0.1 + 0.2
0.30000000000000004


>>> 0.1 + 0.2 == 0.3
False
```

Explanation

The result is False because many decimal fractions cannot be represented exactly in binary floating-point format. Python stores a nearby binary approximation instead.

This small difference can create incorrect totals when thousands of farmer payments are added.

5. Why Float Should Not Be Used for Money

Using float for money can produce small rounding errors. These errors may accumulate when calculating payments for hundreds or thousands of farmers.

Three possible approaches are:

1. Integer Paise

Convert rupees into paise and store the amount as an integer.

Example: Rs. 33.40 = 3340 paise.

Integer addition gives exact results when amounts are represented in whole paise.

2. round() Function

The round() function rounds a number to a specified number of decimal places.

Example:

>>> round(33.456, 2)

33.46

However, rounding a float does not remove the underlying binary representation issue in every calculation.

3. Decimal Module

The decimal module provides decimal arithmetic suitable for financial calculations.

Example:

>>> from decimal import Decimal

>>> Decimal("0.1") + Decimal("0.2")

Decimal('0.3')

For this payment calculator, Decimal created from text values is a suitable choice for accurate money calculations.

6. Complete the Code

The given program contains a TODO section. The total_decimal() function must add the payment amounts accurately using Decimal.

The completed program is provided separately as task02.py.

Expected Output

float : 33400.30000000061

Decimal: 33400.30

False True

Explanation

- The float total contains a small precision error.
- The Decimal total is exactly Rs. 33,400.30.
- 0.1 + 0.2 == 0.3 returns False.
- Decimal("0.1") + Decimal("0.2") == Decimal("0.3") returns True.

7. Changing Decimal(amount) to Decimal(float(amount))

When Decimal(amount) is changed to Decimal(float(amount)), the float approximation is converted into a Decimal value.

This does not restore the original exact decimal value. The small binary floating-point error is carried into the Decimal calculation.

Therefore, Decimal values should be created directly from strings, such as Decimal("33.40"), instead of converting through float.

8. Alternative Concept and Complexity Analysis

Integer Paise Method

Instead of using Decimal, each payment can be converted into integer paise.

Example:

amount_paise = 3340

total_paise = total_paise + amount_paise

After addition, convert the total back into rupees by dividing by 100.

This gives exact addition for values represented in whole paise.

Float with round()

Another approach is to use float and round the total after every addition.

total = round(total + float(amount), 2)

This keeps the displayed total to two decimal places, but it is not guaranteed to provide exact decimal arithmetic.

Complexity Comparison

Method	Time per addition	Extra storage	Exact bank total?

Float	O(1)	O(1)	Not guaranteed
Integer paise	O(1)	O(1)	Yes, for whole-paise amounts
Decimal	O(1) for each fixed-precision operation	O(1)	Yes, when constructed from decimal strings and rounded according to the payment rule

For n payments, all three methods require O(n) time to calculate the total. The total accumulator uses O(1) extra space.

Memory Measurement

The sys.getsizeof() function can be used to compare the memory occupied by individual values. The actual byte counts depend on the Python implementation and version.

The program prints the measurements when executed.

9. Result

Python data types and arithmetic operators were explored using the interpreter. Floating-point precision errors were demonstrated, and the Decimal and integer-paise methods were studied as alternatives. Decimal values created directly from strings are recommended for accurate milk payment calculations.